Pipelines de Dados Tabulares, Datasets e Modelagem Multitarefa

Aula 03 — Ingestão de Dados, Prevenção de Data Leakage, Classificação & Regressão no PyTorch

Instrutor: **Prof. Allan Spadini**

Redes Neurais Profundas | Graduação Faculdade Infnet

Roteiro & Objetivos da Aula

Cinco etapas estratégicas para dominar pipelines supervisionados

01

Integridade & Leakage

Prevenção de contaminação estatística antes do split.

02

Dataset & DataLoader

A anatomia do carregador de dados oficial do PyTorch.

03

Batching & Shuffle

Mini-batches estocásticos e paralelismo com workers.

04

Classificação Multiclasse

Modelagem com CrossEntropyLoss no dataset de E-commerce.

05

Regressão Contínua

Modelagem com MSELoss e métricas MAE, RMSE e R^2 (Insurance).

A Regra de Ouro do Particionamento

Evitando contaminação estatística entre treino, validação e teste

ERRO COMUM: LEAKAGE

Normalizar Antes de Dividir

- Calcular média (μ) e desvio (σ) sobre TODO o dataset.
- A informação do teste contamina as amostras de treino.
- Resulta em perda artificialmente baixa durante o desenvolvimento.
- Falha catastrófica quando o modelo entra em produção real.

PRÁTICA PROFISSIONAL

Split Estrito Primeiro

- Dividir os dados: 70% Treino, 15% Validação, 15% Teste.
- Ajustar os escaladores (.fit_transform) APENAS no Treino.
- Aplicar (.transform) estritamente na Validação e no Teste.
- Garante avaliação cega, realista e sem contaminação.

Simulador Interativo: Detecção de Data Leakage

Compare o comportamento do modelo com vs. sem vazamento de dados

🧪 Sandbox Interativo: Vazamento de Dados no Pré-Processamento

Compare o impacto de escalar features antes vs depois da divisão Treino/Teste

⚠️ Modo Com Vazamento (Errado) | ✅ Modo Robusto (Sem Vazamento)

PASSO 1 Dados Brutos (6 amostras) | **PASSO 2** StandardScaler (No Dataset Inteiro) | **PASSO 3** Métricas & Produção Real

📋 Conjunto completo de dados bancários (4 para treino, 2 para teste):

ID #1 Treino Idade: **25 anos** Renda: **R$ 45.000**	**ID #2** Treino Idade: **42 anos** Renda: **R$ 82.000**	**ID #3** Treino Idade: **31 anos** Renda: **R$ 54.000**
ID #4 Treino Idade: **55 anos** Renda: **R$ 120.000**	**ID #5** Teste (Inédito) Idade: **22 anos** Renda: **R$ 31.000**	**ID #6** Teste (Inédito) Idade: **48 anos** Renda: **R$ 95.000**

Próximo Passo ➔ (Passo 2)

Anatomia da Classe Dataset

Como o PyTorch padroniza o acesso aos dados em memória ou disco

01

__init__(self, X, y)

Recebe os tensores convertidos e armazena os atributos e rótulos.

02

__len__(self)

Retorna a quantidade total de amostras disponíveis: return len(self.X).

03

__getitem__(self, idx)

Indexação direta: recupera o par (amostra_x, alvo_y) na posição idx.

O Papel Estratégico do DataLoader

Por que não treinamos com o dataset inteiro de uma só vez?

BATCH GRADIENT DESCENT

Passar Tudo de Uma Vez

- Exige memória RAM/VRAM gigantesca para carregar tudo.
- Calcula um único gradiente por época completa.
- Convergência extremamente lenta em vales planos.
- Fácil de ficar preso em mínimos locais ou pontos de sela.

MINI-BATCH SGD

DataLoader com Mini-Batches

- Divide os dados em lotes (ex: 32, 64, 128 amostras).
- Atualiza os pesos dezenas de vezes a cada época.
- O ruído estocástico ajuda a escapar de mínimos locais.
- shuffle=True no treino evita viés de ordem de classes.

Visualizador Interativo: O Efeito do Shuffle

Observe a distribuição das classes por mini-batch a cada nova época

🔀 DataLoader Batching & Shuffle Visualizer

Impacto da aleatorização de batches no treinamento de Redes Neurais

✅ shuffle=True (Recomendado Treino) | 🚀 Nova Época (Época 1)

Batch #1	Batch #2	Batch #3
4 amostras	4 amostras	4 amostras
#7 #6 #8 #5	#9 #4 #1 #12	#10 #3 #2 #11
Legítima: 2 / Fraude: 2	Legítima: 2 / Fraude: 2	Legítima: 2 / Fraude: 2

✨ Convergência Suave (SGD com Variância Controlada)

Com shuffle=True, cada batch mistura amostras de todas as classes. Os gradientes apontam para o mínimo global médio de maneira uniforme a cada iteração!

Simulador do Forward Pass Tabular

Acompanhe a propagação dos atributos pelas camadas lineares e ativações

Simulador do Forward Pass da MLP no Dataset Iris

☑ Com ReLU (Não-Linear)

Setosa (Classe 0) | Versicolor (Classe 1) | Virginica (Classe 2)

ENTRADAS X (4)

Sépala Comp: **5.1**
Sépala Larg: **3.5**
Pétala Comp: **1.4**
Pétala Larg: **0.2**

→

CAMADA OCULTA (RELU)

h1:	1.11
h2:	0.83
h3:	3.38

→

SOFTMAX PROBABILIDADES

Setosa (0):	91.9%
Versicolor (1):	1.3%
Virginica (2):	6.8%

Classe Real Alvo: **Setosa (Classe 0)** Cross Entropy Loss: **0.0845**

Classificação vs. Regressão em PyTorch

Ajustando a camada de saída e a função de custo para cada tarefa

CLASSIFICAÇÃO MULTICLASSE

E-commerce Satisfaction (3 Classes)

- Camada de saída: nn.Linear(hidden_dim, 3).
- Saída bruta: Logits reais (sem Softmax manual).
- Função de Perda: nn.CrossEntropyLoss().
- Métricas: Acurácia, Precision, Recall e Matriz de Confusão.

REGRESSÃO CONTÍNUA

Custos Médicos de Seguros (Insurance)

- Camada de saída: nn.Linear(hidden_dim, 1).
- Saída bruta: Escalar contínuo em dólares ($).
- Função de Perda: nn.MSELoss() ou nn.HuberLoss().
- Métricas: MAE, RMSE e Coeficiente R^2.

A Matemática da Entropia Cruzada

Por que o PyTorch não utiliza Softmax explícito na última camada

$$\mathcal{L}_{CE} = -\sum_{c=1}^{C} y_c \log(\hat{p}_c) = -\log\left(\frac{e^{z_y}}{\sum_j e^{z_j}}\right)$$

y_c

Rótulo verdadeiro em one-hot (1 para a classe correta, 0 para as demais).

$\hat{p}_c$

Probabilidade estimada pela rede para a classe c.

z_y

Logit bruto gerado pela camada linear final para a classe correta.

Estabilidade Numérica

O PyTorch usa o Log-Sum-Exp trick para evitar underflow numérico de exponenciais.

Modelando Regressão: O Caso do Seguro Saúde

Prevendo custos anuais de pacientes a partir de características biométricas

ATRIBUTOS DE ENTRADA (X)

Variáveis Demográficas & Saúde

- Idade (Age) e Índice de Massa Corporal (BMI).
- Número de Dependentes (Children).
- Fumante ou Não Fumante (Smoker: yes/no).
- Região geográfica (Northeast, Northwest, etc.).

VARIÁVEL ALVO (Y)

Despesas Médicas em Dólares ($)

- Distribuição altamente assimétrica com cauda longa à direita.
- Média de ~$13.270, mas com pacientes ultrapassando $60.000.
- Exige normalização rigorosa das features de entrada.
- Penalização quadrática (MSE) para erros graves de subestimação.

Métricas Essenciais de Regressão

Avaliando a qualidade de predição além da função de perda de treino

$$MAE = \frac{1}{N}\sum|y - \hat{y}|, \quad RMSE = \sqrt{\frac{1}{N}\sum(y - \hat{y})^2}, \quad R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$

MAE

Erro Médio Absoluto: magnitude média do erro na mesma unidade ($).

RMSE

Raiz do Erro Quadrático Médio: penaliza erros desproporcionais e outliers.

R²

Coeficiente de Determinação: percentual da variância dos dados explicada pelo modelo.

A Necessidade de Baselines Não-Triviais

Como provar cientificamente que sua rede neural realmente aprendeu

BASELINE EM CLASSIFICAÇÃO

Preditor Majoritário (Dummy)

- Chutar sempre a classe mais comum no dataset.
- Exemplo: Se 70% dos clientes são satisfeitos, acurácia baseline = 70%.
- Se sua rede obtiver 72%, ela quase não superou o chute ingênuo!
- Exige cálculo de Precision, Recall e ganho sobre a classe majoritária.

BASELINE EM REGRESSÃO

Preditor da Média Global

- Chutar sempre a média aritmética das despesas ($\bar{y}$).
- Por definição matemática, o preditor da média tem $R^2 = 0.0$.
- Se o modelo obtiver $R^2 \leq 0$, ele é pior do que uma régua fixa!
- Sua rede deve apresentar MAE e RMSE substancialmente inferiores à média.

Quiz de Fixação: Pipelines & Modelagem Tabular

Teste seus conhecimentos sobre Data Leakage, DataLoaders e Funções de Perda

❓ **Quiz de Fixação: Questão 1 de 4**

Pontuação: 0 / 4

1. O que ocorre se aplicarmos o StandardScaler ANTES da divisão train_test_split?

A) Vazamento de Dados (Data Leakage), pois as estatísticas (média/desvio) do teste influenciam o treino.

B) O treinamento fica mais rápido, pois o scaler é executado uma única vez.

C) O modelo aprende melhor, pois vê a distribuição real inteira.

D) O PyTorch dispara um erro de runtime ao criar o DataLoader.

Síntese da Aula & Conexão com o Projeto 1

Checklist de entrega do pipeline fundamental em PyTorch

01

Pipeline Seguro

Split estratificado antes de qualquer ajuste de transformações estatísticas.

02

Dataset & DataLoader

Implementação com __len__, __getitem__, batch_size e shuffle=True no treino.

03

Modelagem Dupla

Domínio de pipelines tanto para classificação multiclasse quanto para regressão.

04

Avaliação Honesta

Métricas mensuradas no teste cego contra baselines documentados.

Próximos Passos: Aula 04

A física interna do treinamento: Estabilidade, Inicialização & Normalização

01

Simetria de Pesos

Por que inicializar pesos com zero impede o aprendizado da rede.

02

Xavier & Kaiming

Calibração da variância das ativações para Sigmoid, Tanh e ReLU.

03

BatchNorm & LayerNorm

Combate ao Internal Covariate Shift e aceleração da convergência.

04

Inverted Dropout

Regularização e eliminação da co-adaptação entre neurônios.